

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) Write a C code to create a process using fork() system call.

b) Read the user input as shell command.

c) Use a system call exec() to execute the shell command.

e) Display the PIDs of parent and child processes.

```
LST@LAPTOP-EK5DCJVE:~# nano task1.c
LST@LAPTOP-EK5DCJVE:~# |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        return 1;
    }
    else if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
        printf("Command execution failed.\n");
        exit(1);
    }
    else {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```

1)Then press ctrl+oenter ctrl+x2)then gcc filename.c -o filename

```
LST@LAPTOP-EK5DCJVE:~# nano task1.c
LST@LAPTOP-EK5DCJVE:~# gcc task1.c -o task1
LST@LAPTOP-EK5DCJVE:~# |
```

3)then ./filename

```
LST@LAPTOP-EK5DCJVE:~# nano task1.c
LST@LAPTOP-EK5DCJVE:~# gcc task1.c -o task1
LST@LAPTOP-EK5DCJVE:~# ./task1
Enter a Linux command: |
```

4)enter the linux command

```
LST@LAPTOP-EK5DCJVE:~# ./task1
Enter a Linux command: ls
Parent PID: 735
Child PID: 756
OSSP file practical_2.c program program.c task1 task1.c teja teja.c
Child process completed.
LST@LAPTOP-EK5DCJVE:~# |
```

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware

Linux Commands Used

1. uname

The uname command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
LST@LAPTOP-EK5DCJVE:~# uname
Linux
LST@LAPTOP-EK5DCJVE:~# |
```

2. lscpu

The lscpu command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
LST@LAPTOP-EK5DCJVE:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:    0-11
Vendor ID:              GenuineIntel
Model name:             13th Gen Intel(R) Core(TM) i5-13420H
CPU family:             6
Model:                  186
Thread(s) per core:     2
Core(s) per socket:     6
Socket(s):              1
Stepping:               2
BogoMIPS:               5222.40
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtsc
                        cp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid
                        sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs i
                        bpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflush
                        opt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_
                        clear serialize flush_l1d arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:     full
Caches (sum of all):
L1d:                    288 KiB (6 instances)
```

3. ps

The ps command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```
LST@LAPTOP-EK5DCJVE:~# ps
  PID TTY          TIME CMD
   502 pts/0        00:00:00 bash
  1289 pts/0        00:00:00 ps
```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
52	root	19	-1	50428	15772	14592	S	0.7	0.2	0:02.20	systemd-journal
1	root	20	0	21836	12900	9556	S	0.0	0.2	0:00.82	systemd
2	root	20	0	3172	2200	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
6	root	20	0	3188	2128	2008	S	0.0	0.0	0:00.00	init
98	root	20	0	25268	6720	5036	S	0.0	0.1	0:00.19	systemd-udev
146	systemd+	20	0	21336	12864	10736	S	0.0	0.2	0:00.15	systemd-resolve
162	systemd+	20	0	91028	7808	6844	S	0.0	0.1	0:00.19	systemd-timesyn
178	root	20	0	4236	2616	2368	S	0.0	0.0	0:00.00	cron
179	message+	20	0	9648	5308	4572	S	0.0	0.1	0:00.38	dbus-daemon
192	root	20	0	17964	8696	7660	S	0.0	0.1	0:00.16	systemd-logind
204	syslog	20	0	222508	5908	4516	S	0.0	0.1	0:00.08	rsyslogd
221	root	20	0	107028	22748	13292	S	0.0	0.3	0:00.09	unattended-upgr
223	root	20	0	3116	1924	1788	S	0.0	0.0	0:00.01	agetty
321	root	20	0	6664	4452	3684	S	0.0	0.1	0:00.00	login
367	root	20	0	20112	11144	9208	S	0.0	0.1	0:00.12	systemd
368	root	20	0	21168	3484	1768	S	0.0	0.0	0:00.00	(sd-pam)
390	root	20	0	6072	5244	3620	S	0.0	0.1	0:00.00	bash
498	root	20	0	3176	1112	980	S	0.0	0.0	0:00.00	SessionLeader
500	root	20	0	3192	1252	1108	S	0.0	0.0	0:00.02	Relay(502)
502	root	20	0	6072	5296	3616	S	0.0	0.1	0:00.02	bash
1106	polkitd	20	0	308224	8004	7096	S	0.0	0.1	0:00.04	polkitd
1362	root	20	0	9316	5604	3396	R	0.0	0.1	0:00.02	top
1377	root	20	0	25272	3836	2116	S	0.0	0.0	0:00.00	(udev-worker)
1378	root	20	0	25272	3836	2116	S	0.0	0.0	0:00.00	(udev-worker)